Cybersecurity Lab 1 · ELTE · Hamdan Al-Jabri · 11 October 2026

Post-quantum KEMs: summary of Part 1

Performance (E1) and correctness (E4) on Apple M4 (ARM64) and Intel i7-1185G7 (x86-64)

69/69
checks pass on both machines
1,332 vectors incl. NIST FIPS 203 (ACVP)

2.2× / 3.5×
ML-KEM-768 cheaper than X25519
full key exchange, M4 / i7

1.6× / 1.7×
hybrid X25519MLKEM768 vs X25519
= its two parts added together

43–841×
conservative KEMs vs X25519
HQC, FrodoKEM, McEliece, level 1 (M4)

25–16,000×
more bytes than X25519
ML-KEM-768: 2,272 B vs 64 B

up to 1,400×
speed-up from AVX2 code
McEliece decaps; NEON only for ML-KEM

1 Why these four KEMs

One KEM from each corner of a 2×2 map, so that the comparison isolates both the *family* (lattice vs. code) and the price of *dropping algebraic structure*:

KEM	family / structure	status (Oct 2026)	role in the study
ML-KEM	lattice, structured (Module-LWE)	NIST FIPS 203 (2024)	the standard; reference point
HQC	code, structured (quasi-cyclic)	NIST selected (2025)	NIST's backup with different maths
FrodoKEM	lattice, unstructured (plain LWE)	ISO/IEC 18033-2 Amd 2 (2026), BSI	ML-KEM without structure
Classic McEliece	code, unstructured (Goppa)	ISO (2026), BSI	most conservative; huge keys

Each at NIST levels 1, 3 and 5 (12 KEMs), against classical **X25519** and the deployed hybrid **X25519MLKEM768**.

2 KEMs, and how Diffie–Hellman becomes one

A **KEM** is three algorithms: $\mathsf{KeyGen}() \to (pk, sk)$; $\mathsf{Encaps}(pk) \to (ct, K)$; $\mathsf{Decaps}(sk, ct) \to K$. Both sides end with the same 32-byte key K after one message each way.

Post-quantum schemes have no efficient Diffie–Hellman, so TLS carries KEMs in its key shares. **Ephemeral X25519 already has the KEM shape**: the client's key pair is KeyGen, the server's key pair + DH is Encaps, and the client's DH is Decaps. We time it that way (**X25519-ECDH**, as in TLS 1.3) and also as **DHKEM** (RFC 9180), which adds HKDF and costs 10–14% more.

The **hybrid** sends ML-KEM's key share followed by X25519's (1,216 / 1,120 B) and concatenates both secrets (64 B).

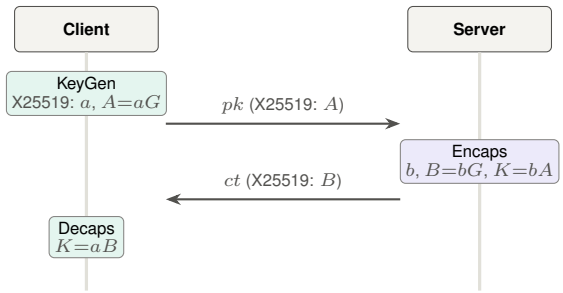


Figure 1. KEM roles, with plain X25519 mapped onto them.

3 Setup and choices

- **Same software on both machines**: Ubuntu 24.04, GCC 13.3, liboqs 0.16.0, OpenSSL 3.5.8; only the CPU differs (M4 in Docker Desktop, i7 in WSL2).
- **Three builds**: liboqs `ref` (portable C), liboqs `opt` (AVX2/NEON), OpenSSL (X25519, ML-KEM, hybrid). Optimised code exists for ML-KEM on both CPUs, for FrodoKEM and McEliece only on x86, for HQC on neither.
- **Timing**: every call timed with the CPU counter, 1,000 calls per operation (McEliece keygen 100), **median** and IQR; per-run calibration, 200 ms warm-up, 30 s cooldown (fanless Air), core clock estimated so results can also be given in cycles.
- **McEliece keygen: 100 samples** (`N_MCE_KEYGEN=100`). One key generation takes 40–500 ms and makes a public key of up to 1 MB, so 1,000 runs would add hours. It is also a try-until-it-works process: it picks a random secret, tries to bring the public matrix into systematic form (succeeds ≈29% of the time) and otherwise starts again. The number of retries is random, so its spread is 55–120% (vs. ∼1% for most operations); 20 samples gave an unstable median, 100 a stable one. This is also why McEliece keygen shows "speed-ups" of 0.95–1.7× on the M4 without any optimised code.

4 E4: correctness

All 69 checks pass on both machines (1,332 vectors and round trips each): liboqs known-answer tests for all 12 KEMs in both builds, NIST ACVP FIPS 203 vectors for ML-KEM (360/360), random round trips, ref↔opt cross-checks, and liboqs↔OpenSSL ML-KEM interoperability. The optimised code computes exactly what the portable code does, on ARM64 and x86-64, so E1 compares like with like.

5 E1: what one key exchange costs

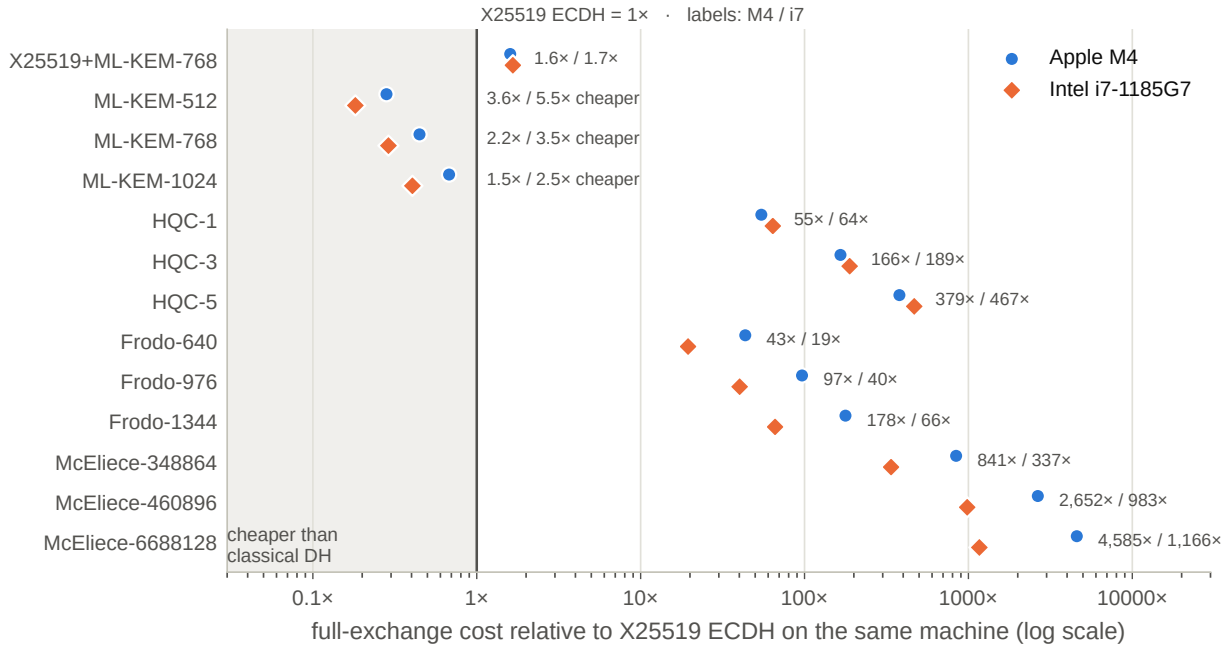


Figure 2. Compute for a full exchange (keygen + encaps + decaps, best build) relative to X25519 ECDH on the same machine.

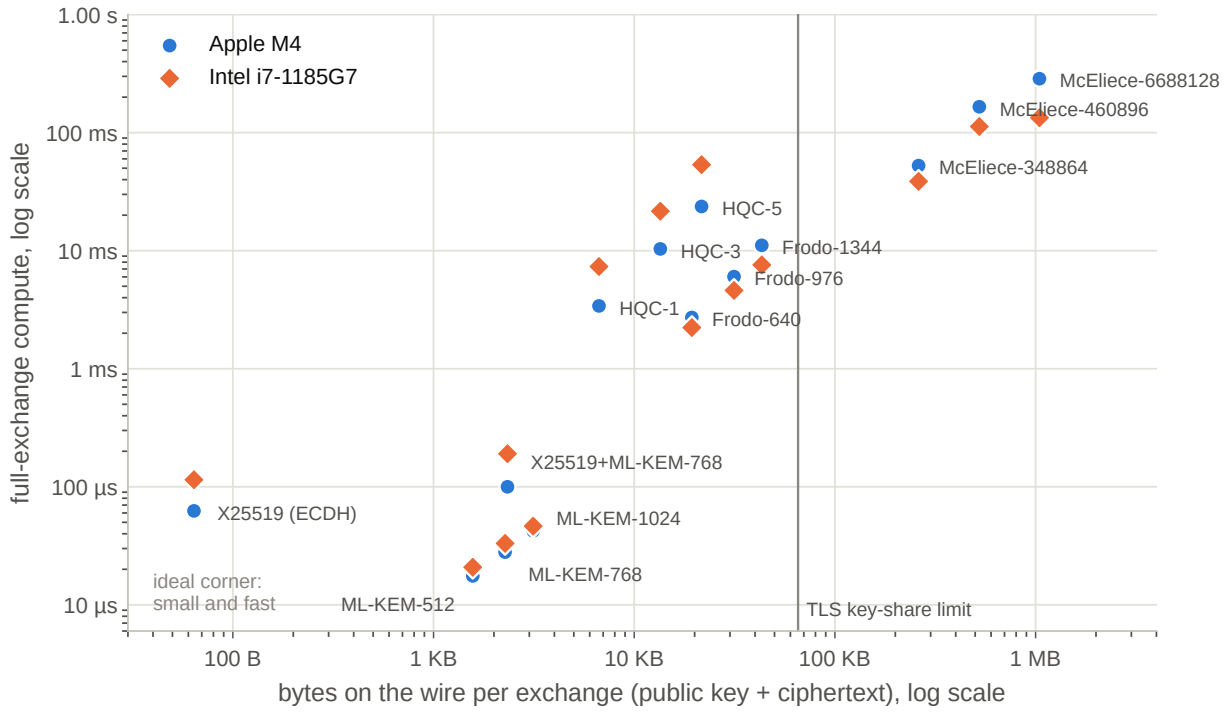


Figure 3. Compute vs. bytes on the wire per exchange. Bottom-left is best; McEliece's public key exceeds the TLS limit.

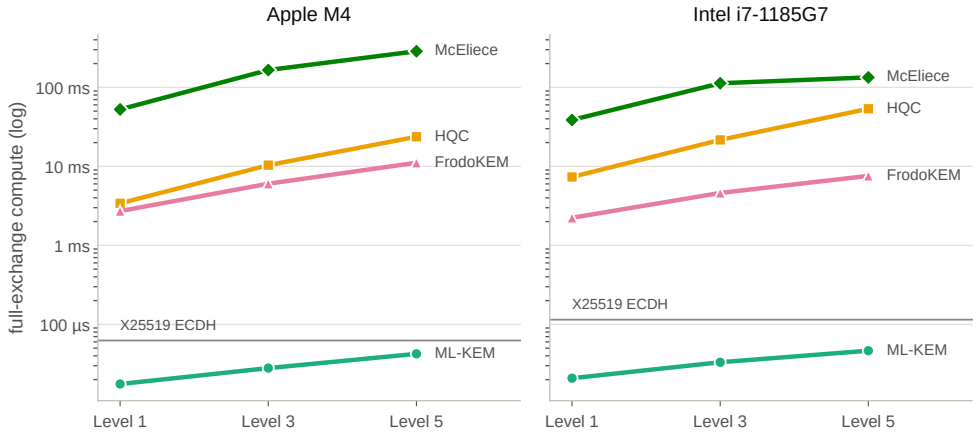


Figure 4. Full-exchange compute from NIST level 1 to 5 (log scale): ML-KEM stays below X25519 even at level 5.

	Apple M4 (NEON)			Intel i7-1185G7 (AVX2)		
	keygen	encaps	decaps	keygen	encaps	decaps
ML-KEM-512	1.6×	1.5×	1.5×	3.2×	3.2×	3.0×
ML-KEM-768	1.7×	1.4×	1.5×	3.2×	3.0×	2.8×
ML-KEM-1024	1.7×	1.4×	1.4×	3.5×	3.1×	2.8×
HQC-1	1.0×	1.0×	1.0×	1.1×	1.0×	1.0×
HQC-3	1.0×	1.0×	1.0×	1.0×	1.0×	1.0×
HQC-5	1.0×	1.0×	0.9×	1.0×	1.0×	1.0×
Frodo-640	1.0×	1.0×	1.0×	2.9×	2.8×	2.8×
Frodo-976	1.0×	1.0×	1.0×	3.1×	3.0×	3.0×
Frodo-1344	1.0×	1.0×	1.0×	3.1×	3.3×	3.6×
McEliece-348864	1.3×	1.0×	1.0×	1.8×	3.6×	511×
McEliece-460896	1.0×	0.9×	1.0×	2.1×	3.1×	574×
McEliece-6688128	1.7×	1.0×	1.0×	3.7×	3.9×	1405×

speed-up = ref time / opt time; darker = larger speed-up (log colour scale); 1.0× = no optimised code for that KEM

Figure 5. Speed-up of optimised (AVX2 / NEON) over portable code; 1.0× = no optimised code for that KEM on that CPU.

6 Key findings

1. **ML-KEM is cheaper than classical X25519** at every level: 2.2× (M4) and 3.5× (i7) at level 3. It pays in bandwidth (25–49× more bytes), not CPU: small 12-bit arithmetic with the NTT vs. X25519's long chain of 255-bit multiplications.
2. **The hybrid costs 1.6–1.7× X25519**, exactly X25519 plus ML-KEM: post-quantum TLS is cheap in compute.
3. **Conservative KEMs cost more in both dimensions**: at level 1 HQC 55–64×, FrodoKEM 19–43×, McEliece 337–841× the compute of X25519, with 4–14× ML-KEM's bytes (McEliece: 0.26–1 MB public keys, too big for TLS).
4. **Implementation matters as much as the algorithm**: AVX2 speeds ML-KEM and FrodoKEM up ∼3× and McEliece decaps 510–1,400×; the M4 has NEON code only for ML-KEM (1.4–1.7×), so it beats the i7 by 2.2× on portable code but loses on FrodoKEM and McEliece. liboqs has no AVX2 HQC: the HQC team's own AVX2 code is 33–49× faster than what we measured.
5. **The i7 lowers its clock for AVX2 code** (3.9 → 3.2 GHz); the fanless M4 stays at 4.0–4.44 GHz with cooldowns. Results are also reported in core cycles for this reason.
6. **Validation**: HQC, McEliece encaps/decaps and the sizes match published figures; ML-KEM and FrodoKEM use fewer cycles than the 2021 spec code. Open: McEliece-348864 keygen is 2.6× the published cycle count.

Limitations: Linux in a VM on both machines; one CPU per architecture; i7 time conversion varies 2.4% between sessions (cycles unaffected). **Next (Part 2):** E2 TLS handshakes under delay and loss, E3 timing side channels. Full details and all tables: the 17-page Part 1 report. Code, data and figures: `github.com/hsh1994-ux/pq-kem-study`. Prepared with help from Claude (Anthropic); design and interpretation are the author's.